

ACKNOWLEDGEMENT

We are very thankful and grateful to our beloved guide, **Mrs. K. ELAKIYA** whose great support in valuable advice, suggestions and tremendous help enabled us in completing our project. She has been a great source of inspiration to us.

We also sincerely thank our Head of the Department; **Dr. R.G. SURESH KUMAR** whose continuous encouragement and sufficient comments enabled us to complete our project report.

We thank all our **Staff members** who have been by our side always and helped us with our project. We also sincerely thank all the lab technicians for their help as in the course of our project development.

We would also like to extend our sincere gratitude and grateful thanks to our **Dr. VIJAYAKRISHNA RAPAKA** for having extended the Research and Development facilities of the department.

We are grateful to our Chairman **Shri. M.K. RAJAGOPALAN**. He has been a constant source of inspiration right from the beginning.

We wish to thank our **family members and friends** for their constant encouragement, constructive criticisms and suggestions that has helped us in timely completion of this project.

ABSTRACT

In the existing system, researchers proposed an artificial intelligence multitier framework with a lightweight classifier for helmetless biker detection. Their method follows a two-stage process: first detecting motorbike riders from traffic footage and then classifying whether they are wearing helmets. By designing a lightweight model, the system reduces computational cost, enabling faster detection with acceptable accuracy. The authors also built custom datasets to train and evaluate the system under different conditions. While this approach achieved promising results, it primarily addressed only helmet detection and was tested under limited scenarios.

The disadvantage of the existing work is that it does not consider broader violations such as triple riding, mobile phone usage, or varying environmental conditions like night and rain. Our proposed project overcomes these gaps by extending detection to multiple unsafe riding behaviours while also integrating robust datasets for diverse scenarios. Additionally, the system will generate automated alerts and send them to authorize institutions or organizations, triggering warning messages to riders. This ensures higher accuracy, real-time enforcement, and practical deployment for improving road safety.

Keywords: Artificial Intelligence, Deep Learning, Helmet Detection, Traffic Safety, Computer Vision.

LIST OF TABLES

TABLE NO.	NAME OF THE TABLE	PAGE NO.
2.6	Literature Survey Comparison	19

LIST OF FIGURES

FIGURE NO.	NAME OF THE FIGURE	PAGE NO
1.2	Deep Learning Architecture	11
1.3	Machine Learning Vs Deep Learning	12
1.4	Neural Network Architecture	13

LIST OF ABBREVIATIONS

- **ML** – Machine Learning
- **DL** – Deep Learning
- **CNN** – Convolutional Neural Network
- **DNN** – Deep Neural Network
- **NPR** – Number Plate Recognition
- **OCR** – Optical Character Recognition
- **ROI** – Region of Interest
- **TP** – True Positive
- **TN** – True Negative
- **FP** – False Positive
- **FN** – False Negative
- **F1** – F1-Score (Harmonic Mean of Precision & Recall)
- **SPP** – Spatial Pyramid Pooling
- **PANet** – Path Aggregation Network
- **RELU** – Rectified Linear Unit
- **FPS** – Frames Per Second
- **GPU** – Graphics Processing Unit
- **YOLO** – You Only Look Once
- **API** – Application Programming Interface

LIST OF SYMBOLS

- **X** – Input image

- $\mathbf{Y}$ – Output label or ground truth
- $\hat{\mathbf{y}}$ – Predicted output
- $\mathbf{w}$ – Weight parameter in a neural network
- $\mathbf{b}$ – Bias term
- $\mathbf{f}(\mathbf{x})$ – Activation function
- $\sum$ – Summation operator
- ∂ – Partial derivative
- ∇ – Gradient operator
- $\otimes$ – Convolution operation
- $\cdot$ – Dot product
- $\|\mathbf{x}\|$ – Norm (magnitude) of vector $\mathbf{x}$
- **TP** – True Positives
- **TN** – True Negatives
- **FP** – False Positives
- **FN** – False Negatives
- α – Learning rate
- $\mathbf{L}$ – Loss function
- **P** – Precision
- **R** – Recall

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	2
	LIST OF TABLES	3
	LIST OF FIGURES	4
	LIST OF ABBREVIATIONS	5
	LIST OF SYMBOLS	6
1.	INTRODUCTIONS	9
	1.0 Overview	9
	1.1 Artificial Intelligence	10
	1.2 Deep Learning	11
	1.3 Deep Learning Vs Machine Learning	12
	1.4 Neural Networks	13
	LITERATURE SURVEY	14
2.	2.1 Survey paper-i	14
	2.2 Survey paper-ii	15
	2.3 Survey paper-iii	16
	2.4 Survey paper-iv	17
	2.5 Survey paper-v	18

	SYSTEM STUDY	20
3.	3.1 Existing System	20
	3.1.1 Limitation of Existing System	21
	3.2 Proposed System	21
	3.2.1 Advantage of Proposed System	22
	3.2.2 Architecture Diagram	22
	SYSTEM REQUIREMENTS	24
4.	4.1 Software Requirements	24
	4.2 Hardware Requirements	24
	4.3 Environment Setup	28
	CONCLUSION	32
5.	5.1 Conclusion	32
	5.2 Phase II of The Project	32
	5.3 Reference	33

CHAPTER I

INTRODUCTION

1.0 OVERVIEW:

In many countries, wearing helmets while riding motorcycles is a legal requirement to ensure the safety of riders. Despite this, the enforcement of helmet laws has been inefficient due to the limitations of manual monitoring by police forces, which are often understaffed and overburdened. CCTV surveillance is used in major cities to address this issue; however, these methods are semi-automated and still rely heavily on human intervention, leading to inconsistencies in enforcement. As motorcycle usage continues to grow, particularly in countries like India, the rate of head injuries and fatalities due to non-compliance with helmet laws has become a pressing concern. This highlights the need for a more efficient, scalable, and automated solution to monitor helmet usage and penalize offenders.

To address this challenge, this paper proposes an automated system that leverages machine learning to detect motorcyclists who are not wearing helmets and retrieve their license plate numbers in real time. Using CCTV camera footage from road junctions, the system employs advanced object detection algorithms to identify helmetless riders. Once detected, the license plate of the motorcycle is captured, and Optical Character Recognition (OCR) is used to extract the license plate number. This information can then be used by authorities to issue penalties or warnings to the offenders. By automating this process, the system not only reduces the dependency on human resources but also ensures consistent and accurate enforcement of helmet laws.

The integration of machine learning in transportation systems has the potential to significantly enhance road safety. Automated helmet detection and license plate recognition systems serve as a practical example of how technology can be applied to improve compliance with traffic regulations. Beyond enforcement, such systems generate valuable data that policymakers can use to make informed decisions about road safety measures. However, implementing these systems comes with challenges. Factors like lighting conditions, weather, and the distance between the camera and the motorcycle can impact the accuracy of detection and recognition algorithms. Additionally, substantial investments are needed to establish the necessary infrastructure and maintain such systems effectively.

Despite these challenges, the benefits of these automated systems are substantial, particularly in developing countries where road safety remains a critical concern. By reducing manual intervention, improving enforcement, and providing actionable insights, these systems can help reduce fatalities and injuries caused by motorcycle accidents. Continued research and development in this domain will further

refine these technologies, making them more reliable and cost-effective. As such systems gain wider adoption, they have the potential to become an integral part of transportation safety frameworks worldwide, paving the way for safer roads and more efficient traffic management.

1.1 ARTIFICIAL INTELLIGENCE

Artificial Intelligence (AI) is the simulation of human intelligence in machines that are programmed to think and act like humans. Learning, reasoning, problem-solving, perception, and language comprehension are all examples of cognitive abilities. Artificial intelligence is a method of making a computer, a computer-controlled robot, or a software think intelligently like the human mind. AI is accomplished by studying the patterns of the human brain and by analyzing the cognitive process. The outcome of these studies develops intelligent software and systems [6].

Artificial Intelligence (AI) is the theory and development of computer systems capable of performing tasks that historically require human intelligence, such as recognizing speech, making decisions, and identifying patterns. AI is an umbrella term that encompasses a wide variety of technologies, including machine learning, deep learning, and natural language processing (NLP). Although the term is commonly used to describe a range of different technologies in use today, many disagree on whether these constitute artificial intelligence. Instead, some argue that much of the technology used in the real world today actually constitutes highly advanced machine learning that is simply a first step towards true artificial intelligence, or “general artificial intelligence” (GAI).

1.1.1 Types of Artificial Intelligence

Artificial Intelligence can be divided into two different categories: weak and strong.

- **WEAK AI**

Weak Artificial Intelligence embodies a system designed to carry out one job. Weak AI systems include video games such as the chess example from above and personal assistants such as Amazon's Alexa and Apple's Siri. You ask the assistant a question, and it answers it for you.[10].

- **STRONG AI**

Strong Artificial Intelligence systems are systems that carry on tasks considered to be human-like. These tend to be more complex and complicated systems. They are programmed to handle situations in which they may be required to problem solve without having a person intervene. These kinds of systems can be found in applications like self-driving cars or in hospital operating rooms [10].

1.2 DEEP LEARNING

According to Geoffrey Hinton (considered as the father of Deep Learning), deep learning (DL) allows computational models that are composed of multiple processing layers to learn representations of data with multiple levels of abstraction. Deep learning is a type of technology that allows computers to simulate how our brains work. It is a subset of machine learning, a branch of artificial intelligence [3]. In deep learning, a computer model learns to perform classification tasks directly from images, text, or sound. Deep learning models can achieve state-of-the-art accuracy, sometimes exceeding human-level performance. Deep learning uses artificial neural networks that consist of multiple layers of nodes, also known as neurons, that can learn from large amounts of data. Deep learning can process unstructured data, such as images and text, and automatically extract important features and patterns from the data [3]. Deep learning can also perform different types of learning, such as supervised, unsupervised, and reinforcement learning. Deep learning is the technology behind many popular AI applications, such as chatbots, virtual assistants, self-driving cars, and more.

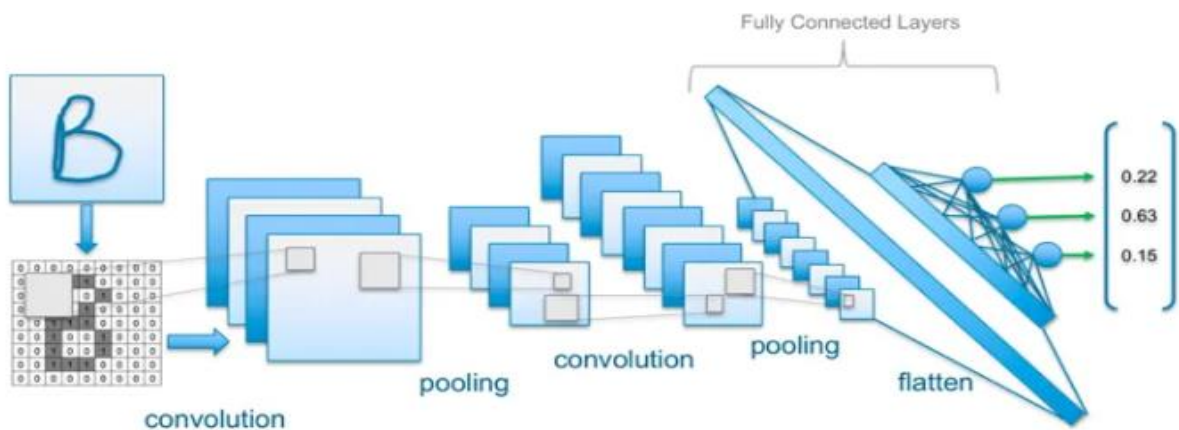


Figure 1.2. Deep Learning Architecture

1.3 DEEP LEARNING VS MACHINE LEARNING

Machine Learning and Deep Learning are revolutionary fields in the computer and information sciences area and are used widely in business applications. Machine Learning is an approach to train computers or machines to learn from past data or behavior so it can determine future data or behavior. Deep Learning is a branch of Machine Learning where the Artificial Neural Network techniques and algorithms are used to train and learn. Due to the large volume of data points generated daily, there is a great need for the proper use of data and data management using the major techniques [11]. Another key difference is deep

learning algorithms scale with data, whereas shallow learning converges. A key advantage of deep learning networks is that they often continue to improve as the size of your data increases. The difference in working of machines and deep learning is shown in figure 1.3.

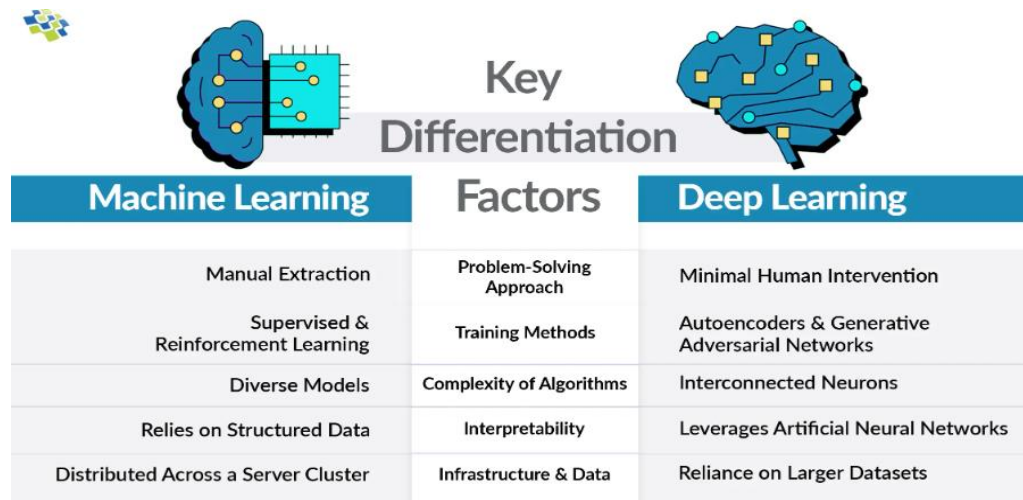


Figure 0.3. Machine Learning Vs Deep Learning

In machine learning, a programmer must intervene directly in the action for the model to come to a conclusion. In the case of a deep learning model, the feature extraction step is completely unnecessary. The model would recognize these unique characteristics of a car and make correct predictions. Deep Learning models tend to increase their accuracy with the increasing amount of training data, where's traditional machine learning models such as SVM and Naive Bayes classifier stop improving after a saturation point [11].

1.4 NEURAL NETWORKS

Deep learning algorithms attempt to draw similar conclusions as humans would by continually analyzing data with a given logical structure. To achieve this, deep learning uses a multi-layered structure of algorithms called neural networks. The design of the neural network is based on the structure of the human brain. Just as we use our brains to identify patterns and classify different types of information, neural networks can be taught to perform the same tasks on data [4].

Neural networks, also known as artificial neural networks (ANNs) or artificially generated neural networks (SNNs) are a subset of machine learning that provides the foundation of deep learning techniques. Their name and form are inspired by the human brain, and they replicate the way real neurons communicate

with one another. Artificial neural networks (ANNs) are massively parallel systems comprised of a huge number of interconnected basic processors [4]. The Neural network architecture is shown in figure 1.4.

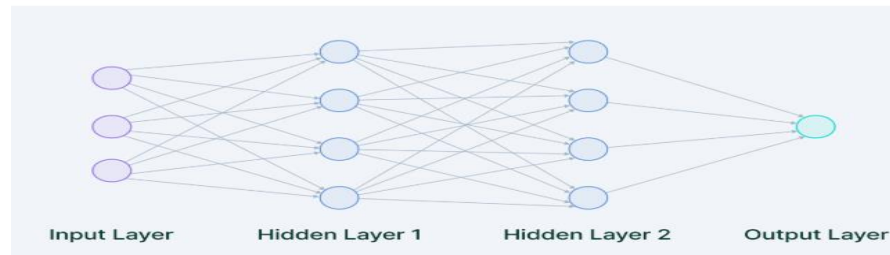


Figure 0.4. Neural Network Architecture

The individual layers of neural networks can also be thought of as a sort of filter that works from gross to subtle, increasing the likelihood of detecting and outputting a correct result. The human brain works similarly. Whenever we receive new information, the brain tries to compare it with known objects. The same concept is also used by deep neural networks.

With neural networks, we can group or sort unlabeled data according to similarities among the samples in this data. Or in the case of classification, we can train the network on a labeled data set in order to classify the samples in this data set into different categories [12].

CHAPTER II

LITERATURE SURVEY

TITLE 1: Automated detection of bicycle helmets using deep learning

AUTHORS: Siebert F.W., Riis C., Janstrup K.H., Lin H., et al.

YEAR: 2024

DESCRIPTION:

This study presents a computer vision approach to automatically detect bicycle helmets from roadside camera footage in Copenhagen. A dataset of ~4,000 cyclists, supplemented with web images, was annotated for active cyclist detection and helmet use. The authors trained and tested deep learning models to estimate helmet-wearing rates, comparing results against manual human observations. The system demonstrated near-human accuracy when tested on the same site, with a minimal underestimation of 0.52%.

However, accuracy dropped when applying models trained at one site to another, highlighting the challenges of cross-site generalization in real-world deployments.

CONCLUSION:

Automated helmet-use detection is feasible and highly accurate under controlled conditions. However, cross-site transfer remains a limitation, and improving robustness across diverse environments is crucial for large-scale deployment.

ADVANTAGES:

- Provides near-human accuracy in detecting helmet usage under controlled conditions.
- Reduces the need for manual observation, saving time, labor.
- Uses real-world roadside camera data, making the approach practical.

DISADVANTAGES:

- Performance drops significantly when applied to different locations (poor cross-site generalization).
- Requires large, annotated datasets, which are time-consuming to prepare.
- Accuracy may be affected by environmental factors such as lighting and camera angle

TITLE 2: Effect of helmet use on maxillofacial injuries: a systematic review and metanalysis

AUTHORS: Stassen H.S., Atalik T., Haagsma J.A., Wolvius E.B., et al.

YEAR: 2024

DESCRIPTION:

This paper systematically reviews and meta-analyses studies examining the relationship between helmet use and maxillofacial injuries in bicycle and scooter crashes. Fourteen studies were included, of which eleven contributed to the quantitative synthesis. The pooled odds ratio ($OR \approx 0.68$) indicates that helmets significantly reduce the risk of maxillofacial trauma. The included studies were predominantly observational, with variable quality ratings. The review emphasizes that while evidence supports protective effects, research is limited by heterogeneity, older datasets, and lack of focus on e-bikes and e-scooters, which have unique injury patterns and growing urban prevalence.

CONCLUSION:

Helmets provide significant protection against maxillofacial injuries. However, higher-quality and more recent studies, particularly addressing e-bikes and e-scooters, are needed to strengthen evidence and guide policy.

ADVANTAGES:

- Strong statistical evidence showing helmets reduce facial injuries.
- Combines multiple studies, increasing reliability of conclusions.
- Useful for policymaking and safety regulations.

DISADVANTAGES:

- Most included studies are observational, which may introduce bias.
- Data heterogeneity makes comparisons difficult.
- Limited focus on modern transport modes like e-bikes and e-scooters.

TITLE 3: AI-Based Helmet Violation Detection for Traffic Management System

AUTHORS: Said Y., Alassaf Y., Ghodhbani R., Alsariera Y.A., et al.

YEAR: 2024

DESCRIPON:

This work introduces an AI-driven helmet violation detection framework aimed at improving road safety and assisting traffic enforcement authorities. The system is built on a modified Perspective Net model with Efficient Net v2 as the backbone, balancing detection accuracy and computational efficiency for real-time surveillance. It is evaluated using public traffic datasets and demonstrates a 95.2% detection accuracy, with reduced model complexity compared to conventional deep learning approaches. The authors highlight scalability in dense urban traffic, capability for real-time operation, and potential for integration into broader intelligent transportation systems.

CONCLUSION:

The study validates the feasibility of using lightweight deep learning models for helmet violation detection in real-world traffic conditions. It emphasizes scalability and efficiency, making the system suitable for real-time deployment in smart cities

ADVANTAGES:

- High detection accuracy (95.2%) with low computational cost.
- Suitable for real-time traffic surveillance systems.
- Scalable for dense urban traffic environments.

DISADVANTAGES:

- Performance may vary across different traffic scenarios.
- Depends heavily on the quality of surveillance cameras.
- May struggle in extreme weather or low-visibility conditions

TITLE 4: DetectNet_v2 + YOLOv8 for helmet violation detection and license plate extraction

AUTHORS: Deshpande U.U., Goh G.K.O.M., Araujo S.D.C.S., et al.

YEAR: 2025

DESCRIPTION:

This paper proposes a two-stage helmet violation detection pipeline tailored for Indian traffic conditions. In the first stage, DetectNet_v2 with a ResNet18 backbone detects motorcycle riders, while the second stage employs YOLOv8 to classify helmet use and detect license plates. An OCR module extracts license numbers for enforcement. A custom dataset of ~1,715 images captured in varied lighting and traffic conditions is used for training and evaluation. The system achieves impressive accuracy: helmet detection (98.56%) and license plate recognition (97.6%), with competitive inference times for near Performance scenes. real-time application in urban surveillance systems.

CONCLUSION:

The two-stage pipeline demonstrates robust performance for helmet violation detection and license plate recognition. Its strong accuracy and practical orientation highlight its suitability for deployment in Indian smart-city surveillance networks.

ADVANTAGES:

- Very high accuracy in helmet detection and license plate recognition.
- Designed specifically for Indian traffic conditions.
- Enables automated enforcement through license plate extraction.

DISADVANTAGES:

- Two-stage architecture increases system complexity
- Requires higher computational resources compared to single-stage models.

TITLE 5: Smart Road Surveillance: Helmet Detection and Triple Ride Detection Using CNN and YOLO

AUTHORS: Ranganayaki J., Poojitha V., Dikshith Y.

YEAR: 2025

DESCRIPTION:

This engineering-oriented paper presents a smart road surveillance system combining YOLO and CNN for detecting helmet violations and triple-riding cases on motorcycles. The system includes license plate recognition using Tesseract OCR, enabling automatic identification of violators. The implementation is tailored for resource-constrained environments, focusing on practical deployment in developing regions. The authors describe the system architecture, dataset preparation, and provide screenshots of real-world tests. While performance metrics are not as comprehensive as in research-focused works, the emphasis is on integrating multiple violation detection features into a functional prototype system.

CONCLUSION:

The proposed system demonstrates the integration of helmet and triple-ride detection into a single surveillance framework. Its strength lies in practical feasibility and low-cost deployment, though further validation with larger datasets is needed.

ADVANTAGES:

- Detects multiple violations (helmet and triple riding) in one system.
- Low-cost and suitable for resource-constrained environments.
- Practical, real-world oriented implementation.

DISADVANTAGES:

- Limited evaluation metrics and experimental validation.
- Accuracy may be lower compared to advanced DL models
- Smaller or unspecified datasets reduce reliability.

2.6 LITERATURE SURVEY COMPARISION:

S.NO.	TITLE	TECHNIQUE USED	CONCLUSION
1.	AI-Based Helmet Violation Detection for Traffic Management System	Optimized PerspectiveNet with EfficientNet v2 backbone, deep learning-based real-time helmet violation detection.	We conclude that architecture optimization enables accurate real-time traffic monitoring, guiding our project toward efficient and scalable safety applications.
2.	Computer-vision based automatic rider helmet violation detection and vehicle identification in Indian smart city scenarios using NVIDIA TAO toolkit and YOLOv8	Two-step approach: DetectNet_v2 (ResNet18 + TAO) for rider detection, YOLOv8 for helmet + number plate detection, OCR for plate recognition; Custom dataset of 1,715 images	We conclude that combining multi-model deep learning improves reliability. This supports our project vision to use a multi-model approach for accurate detection, even in complex real-time traffic scenarios.
3.	Automated detection of bicycle helmets using deep learning	Computer vision + state-of-the-art object detection algorithms trained on annotated cyclist datasets, real-world traffic video analysis.	We conclude that helmet detection can scale beyond motorbikes to bicycles, proving our project can extend to multiple road user categories for broader safety enforcement.

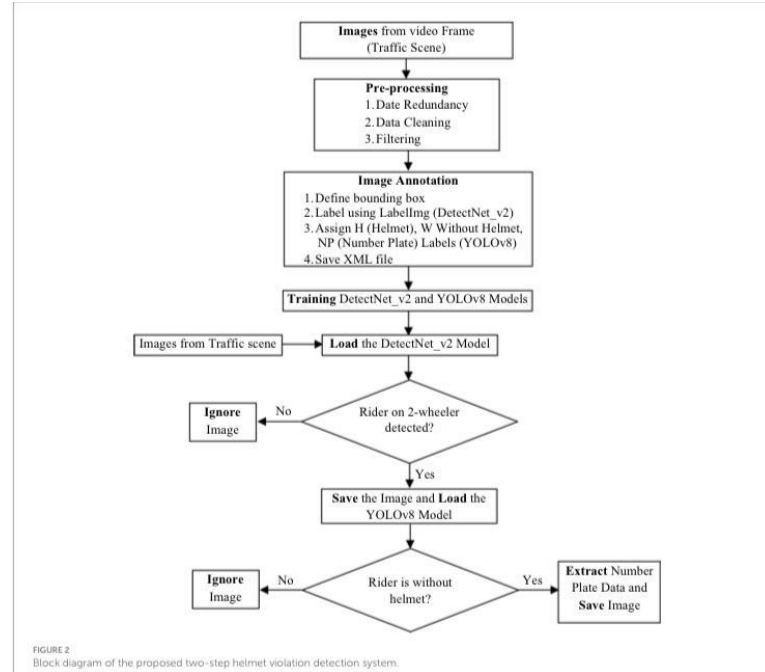
4.	Smart Road Surveillance: Helmet Detection and Triple Ride Detection Using CNN and YOLO	YOLOv5 for motorcycle, rider, and plate detection; CNN for helmet/no-helmet classification; OCR (Tesseract) for license plate recognition; real-time triple ride detection	We conclude that integrating multiple detection tasks (helmet, license plate, triple ride) into a unified system enhances road safety enforcement.
5.	An AI multitier system with lightweight classifier for automated helmetless biker detection	Custom CNN model BikeNet-12, datasets with helmet/no-helmet/safe-unsafe helmet classes, YOLOv8 for object detection in varied conditions.	We conclude that lightweight models can deliver high accuracy with less computation, highlighting efficiency as a key factor for our system's real-world deployment.

CHAPTER III

SYSTEM STUDY

3.1 EXISTING SYSTEM

In the existing system, helmet violation detection is largely dependent on manual monitoring by traffic police or on basic image processing techniques. Traffic officers are required to continuously observe CCTV footage or roadside camera feeds and visually identify riders who are not wearing helmets. This approach is highly time-consuming and demands significant human effort, making it inefficient for monitoring large traffic volumes. Moreover, manual observation is prone to human error, fatigue, and inconsistency, which can lead to missed violations or incorrect enforcement actions. Some conventional systems attempt to automate helmet detection using simple image processing methods based on color, shape, or edge detection.



While these techniques offer partial automation, they lack robustness and perform poorly in real-world conditions. Variations in helmet color and design, changes in lighting conditions such as night-time or shadows, and complex or cluttered backgrounds significantly reduce their accuracy. These systems also struggle with occlusions, camera angle variations, and low-resolution video feeds, making them unreliable for practical deployment. Overall, existing helmet violation detection systems lack full automation, real-time processing capability, and high accuracy. Their dependence on manual intervention and fragile image processing rules limits scalability and effectiveness. These limitations clearly highlight the need for an advanced deep learning-based approach that can automatically learn discriminative features, operate reliably under diverse conditions, and provide efficient, real-time helmet violation detection for modern traffic management systems.

3.1.1 LIMITATION OF THE EXISTING SYSTEM

In the existing system, helmet violation detection is primarily carried out through manual monitoring by traffic police or by using basic image processing techniques. Traffic personnel observe CCTV footage or roadside camera feeds and visually identify riders who are not wearing helmets. This method is highly time-consuming, labor-intensive, and prone to human errors caused by fatigue, distraction, or subjective judgment. To reduce manual effort, some traditional systems apply simple image processing approaches based on predefined color, shape, or edge features to detect helmets. However, these techniques lack robustness and often fail in real-world conditions such as poor lighting, night-time scenarios, varying helmet designs, camera angle changes, and complex backgrounds. Additionally, they are unable to handle

occlusions or crowded traffic environments effectively. As a result, existing systems lack automation, real-time processing capability, and reliable accuracy. These limitations restrict scalability and make them unsuitable for modern traffic surveillance needs, emphasizing the necessity for more advanced and intelligent detection approaches.

3.2 PROPOSED SYSTEM

The proposed system overcomes the limitations of existing helmet violation detection methods by adopting a fully automated deep learning-based framework built on YOLOv12, which offers improved accuracy, faster inference, and better generalization across real-world traffic conditions. YOLOv12 is utilized to precisely detect helmet usage by learning complex visual features, enabling reliable differentiation between helmeted and non-helmeted riders even in challenging scenarios such as varying lighting conditions, different helmet styles, dense traffic, and diverse camera angles. The system is trained using robust and diverse datasets to ensure consistent performance in both urban roads and highways. Live video streams from CCTV cameras are processed in real time, allowing instant identification of helmet violations without the need for continuous human supervision. When a rider without a helmet is detected, the system automatically triggers alert mechanisms by notifying authorized institutions such as traffic police departments and issuing warning notifications to riders. This immediate response supports proactive enforcement and increases rider awareness, encouraging compliance with helmet regulations. The automated workflow significantly reduces human error, monitoring effort, and enforcement delays, making the system suitable for large-scale and continuous deployment. By combining high-accuracy detection, real-time processing, and automated alert generation, the proposed system ensures efficient enforcement and practical implementation. Overall, the system enhances traffic monitoring capabilities, strengthens helmet law enforcement, and contributes to reducing accident-related injuries and fatalities, thereby improving overall road safety.

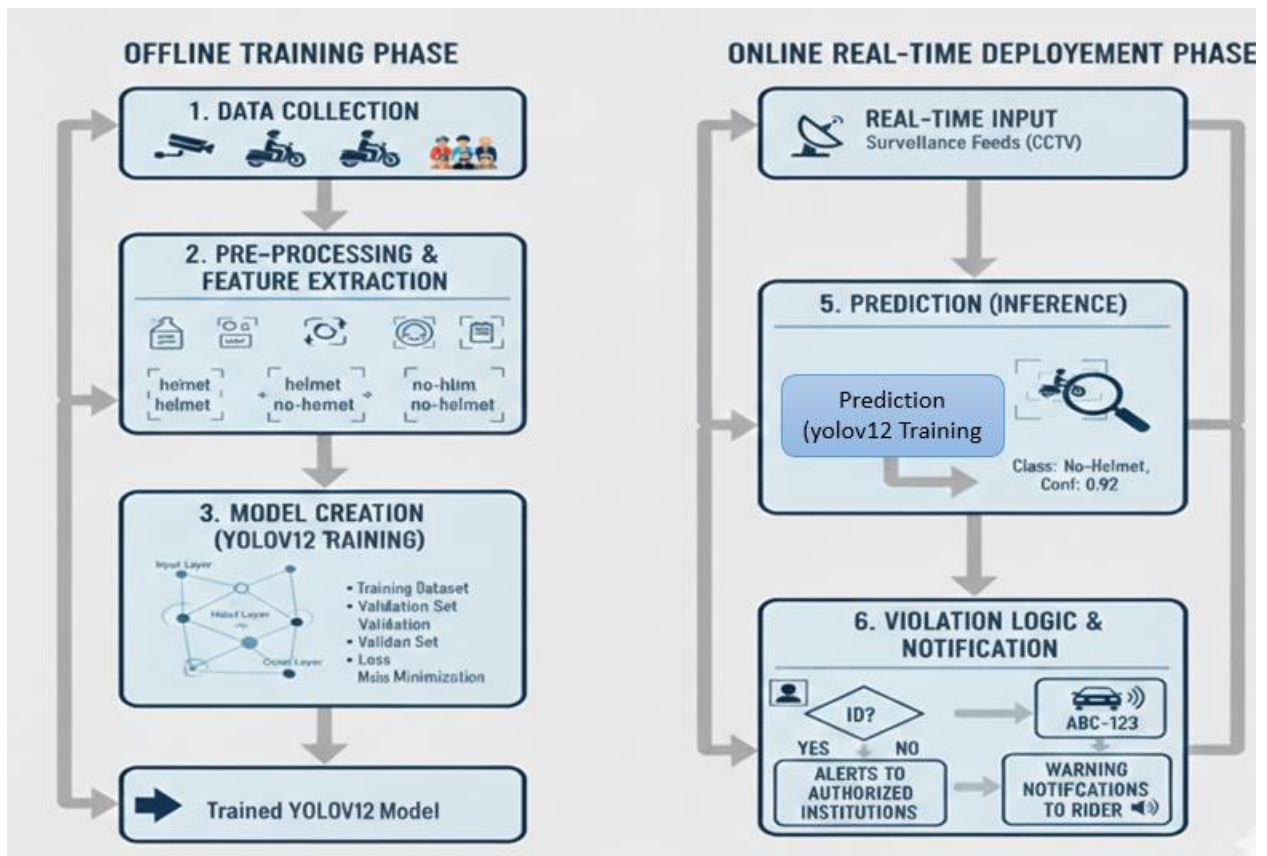
3.2.1 ADVANTAGE OF THE PROPOSED SYSTEM:

- Provides highly accurate helmet detection using advanced deep learning techniques, ensuring reliable performance across diverse real-world traffic conditions.
- Enables real-time processing of live CCTV video streams, allowing instant identification of helmet violations without manual intervention.
- Automatically generates alerts and warning notifications, improving enforcement efficiency and encouraging rider compliance.

- Reduces human effort, operational cost, and errors associated with manual monitoring by traffic authorities.
- Scalable and practical for large-scale deployment in urban and highway traffic surveillance systems to enhance overall road safety.

3.2.3 ARCHITECTURE DIAGRAM:

The architecture diagram illustrates a two-phase framework consisting of an offline training phase and an online real-time deployment phase for helmet violation detection. In the offline training phase, the process begins with data collection, where images and videos of motorcyclists under different conditions are gathered. These data are then passed through a pre-processing and feature extraction stage, where relevant features such as helmet and no-helmet regions are prepared and annotated. These data are then passed through a pre-processing and feature extraction stage, where relevant features such as helmet and no-helmet regions are prepared and annotated.



Next, model creation is carried out using YOLOv12 training, where the dataset is divided into training and validation sets, and the model learns to minimize detection loss through iterative optimization. This phase results in a trained YOLOv12 model capable of accurately distinguishing helmeted and non-helmeted riders. In the online real-time deployment phase, live surveillance inputs from CCTV cameras are fed into the trained model for prediction (inference). The system classifies each detected rider in real time with confidence scores. When a no-helmet violation is detected, violation logic is applied to verify identification

details, and the system automatically generates alerts to authorized institutions and sends warning notifications to riders. This integrated architecture ensures continuous monitoring, real-time enforcement, and efficient road safety management.

CHAPTER IV

SYSTEM REQUIREMENTS

4.1 SOFTWARE REQUIREMENTS

- OS: Windows 10 (or above)
- LANGUAGE: Python 3.11
- Operating System: Windows 10 and above
- Coding language: Python 3.10.2 and above
- Visual studio
- Front End: HTML, CSS, JavaScript
- Back End: SQL lite
- Tensor flow

4.2 HARDWARE REQUIREMENTS

- **PROCESSOR:** Intel(R) core (TM) i5-10thGEN (or above)
- **RAM:** 4 GB (or above)
- **PROCESSOR SPEED:** 2GHz
- **HARD DISK:** 512 GB SSD (or above)
- **KEYBOARD:** Standard Windows Keyboard
- **MOUSE:** 2 or 3 button mouse.

4.2.1 PYTHON:

Python is an interpreted, high-level, general-purpose programming language. Created by Guido van Rossum and first released in 1991, Python's design philosophy emphasizes codereadability with its notable use of significant whitespace. Its language constructs and object-oriented approach aim to help programmers write clear, logical code for small and large-scale projects.

Python is dynamically typed, and garbage is collected. It supports multiple programming paradigms, including procedural, object-oriented, and functional programming. Python is often described as a "batteries included" language due to its comprehensive standard library.

Python is one of those rare languages which can claim to be both simple and powerful. You will find yourself pleasantly surprised to see how easy it is to concentrate on the solution to the problem rather than the syntax and structure of the language you are programming in. Python is an interpreted high-level and general-purpose programming language.

Python's design philosophy emphasizes code readability with its notable use of significant indentation. Its language constructs and object-oriented approach aim to help programmers write clear, logical code for small and large-scale projects. Python is dynamically-typed and garbage-collected. It supports multiple programming, including structured, object-oriented, and functional programming. Python is often described language due to its comprehensive standard library. It supports multiple programming including structured, object-oriented, and functional programming.

4.2.2 TENSORFLOW & KERAS:

TensorFlow is a free and open-source software library for machine learning and artificial intelligence. It can be used across a range of tasks but has a particular focus on training and inference of deep neural networks. TensorFlow was developed by the Google Brain team for internal Google use in research and production. TensorFlow can be used in a wide variety of programming languages, including Python, JavaScript, C++, and Java. TensorFlow serves as the core platform and library for machine learning. TensorFlow's APIs use Keras to allow users to make their own machine learning models. In addition to building and training their model, TensorFlow can also help load the data to train the model and deploy it using TensorFlow Serving. Auto Differentiation is the process of automatically calculating the gradient vector of a model with respect to each of its parameters. With this feature, TensorFlow can automatically compute the gradients for the parameters in a model, which is useful to algorithms such as backpropagation which require gradients to optimize performance. Keras is an open-source software library that provides a Python interface for artificial neural networks. Keras acts as an interface for the TensorFlow library. Keras contains numerous implementations of commonly used neural-network building blocks such as layers, objectives, activation functions, optimizers, and a host of tools to make working with image and text data easier to simplify the coding necessary for writing deep neural network code. The code is hosted on GitHub, and community support forums include the GitHub issues page, and a Slack channel. Keras has support for convolutional and recurrent neural networks. It supports other common utility layers like dropout, batch

normalization, and pooling. Keras allows users to produce deep models on smartphones (iOS and Android), on the web, or on the Java Virtual Machine.

4.2.3 SQLITE3:

SQLite3 is a lightweight, open-source, and self-contained database management system used in the Real Time Helmet Violation Detection project to store and manage application data efficiently. Unlike traditional databases such as MySQL or PostgreSQL, SQLite does not require a separate server and is embedded directly into the application, making it fast and easy to use. In this project, SQLite3 is used to store important information such as detected helmet violation records, vehicle details, image paths, date and time of violations, and system logs. The entire database is stored in a single file, which simplifies data storage and improves portability. SQLite3 supports essential SQL features like tables, indexes, transactions, and queries, ensuring reliable data handling. It is fully ACID compliant, which guarantees data consistency and safety even during system crashes or power failures. Due to its small size, low memory usage, and zero configuration, SQLite3 is well suited for real-time computer vision applications like helmet violation detection. It is especially useful in systems running on limited hardware or edge devices. Overall, SQLite3 provides a simple, reliable, and efficient data storage solution for the Real Time Helmet Violation Detection system, making it ideal for storing and retrieving violation data without complex database management.

4.2.4 FLASK:

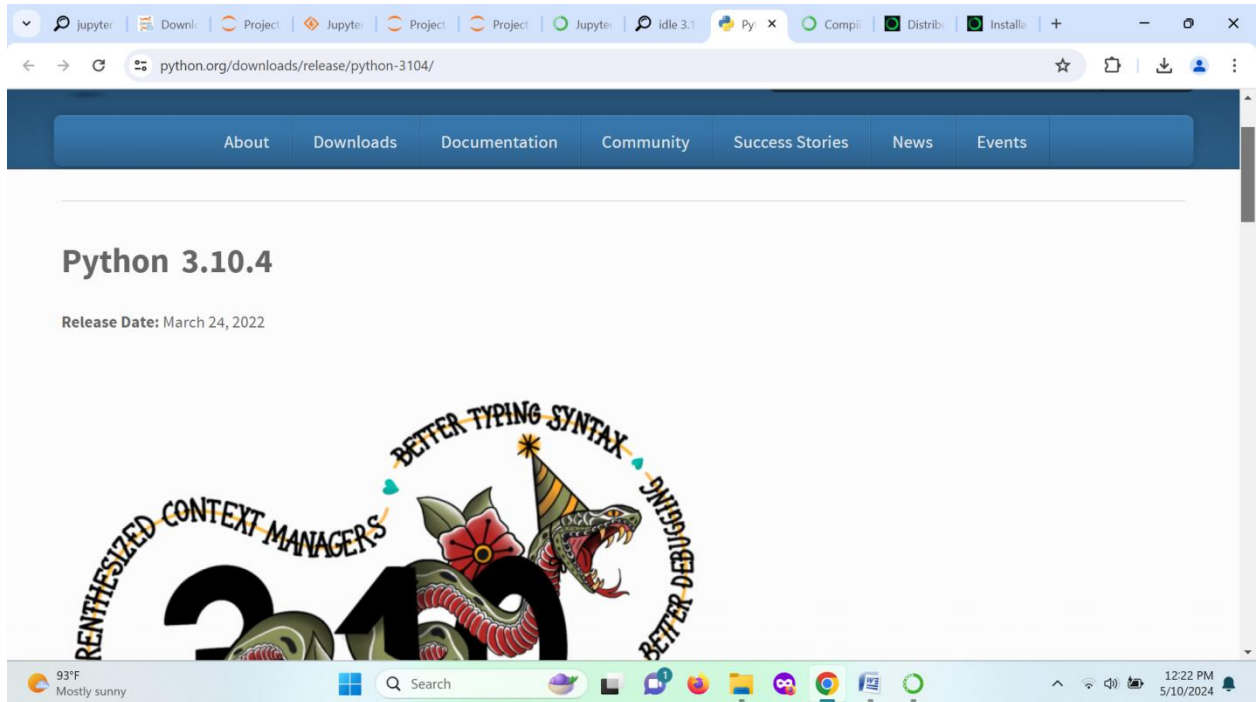
Flask is a lightweight, open-source web framework written in Python that is widely used for developing web applications and RESTful APIs. Designed with simplicity and flexibility in mind, Flask follows the microframework philosophy, meaning it provides the essential tools needed to build web applications without imposing dependencies or a rigid project structure. This makes it highly adaptable for both small-scale projects and large, complex applications. At its core, Flask includes a built-in development server, URL routing, and a templating engine called Jinja2, which allows developers to dynamically generate HTML pages.

It also supports request handling, session management, and error handling, making it a complete yet minimalistic framework. One of Flask's key advantages is its modularity: developers can easily extend its functionality using third-party libraries or integrate it with various databases such as SQLite, MySQL, or PostgreSQL. This modular approach allows for rapid prototyping while maintaining the flexibility to scale applications as requirements grow. Flask is particularly popular for developing RESTful APIs because it

provides straightforward request and response handling, JSON support, and the ability to define clear endpoints. Additionally, Flask's simplicity and minimalism make it an excellent learning platform for beginners in web development, as it emphasizes understanding the fundamentals without abstracting too much functionality. Despite its lightweight design, Flask is production-ready and can handle complex applications when combined with extensions for authentication, form validation, caching, and more. Its active community ensures continuous development, support, and availability of numerous plugins and extensions, making it a robust choice for web development projects. Overall, Flask strikes a balance between simplicity and extensibility, offering developers the freedom to structure applications as they see fit while providing essential tools for building modern, scalable, and maintainable web applications.

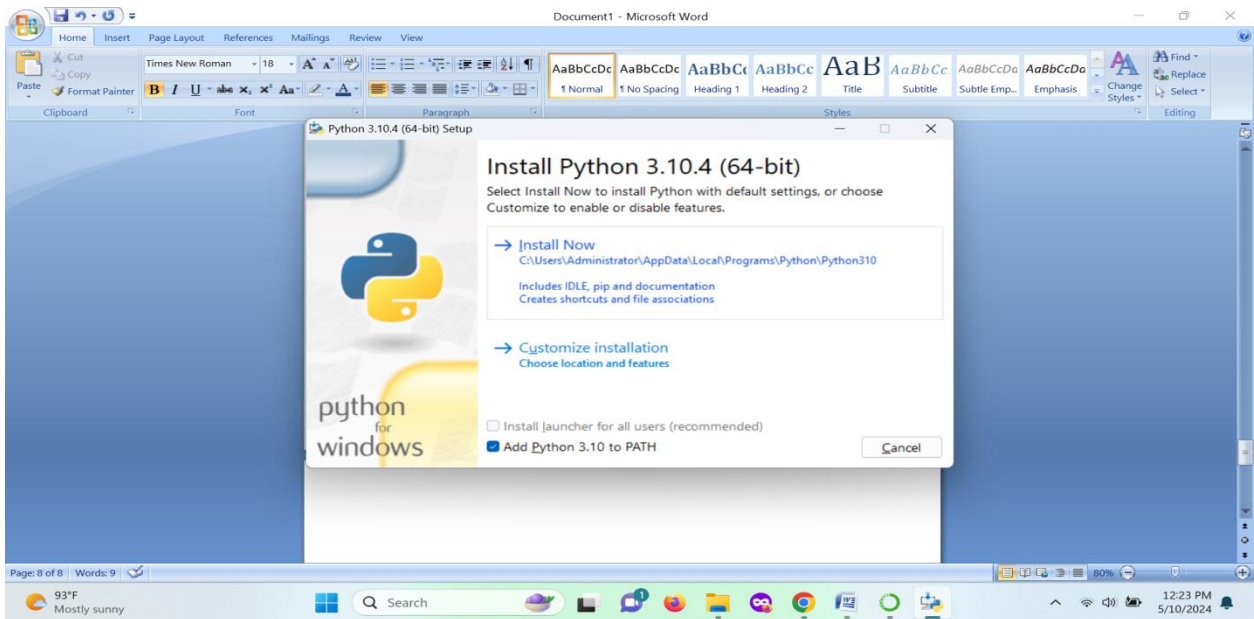
4.3 ENVIRONMENTAL SETUP:

4.3.1 DOWNLOAD IDLE PYTHON:



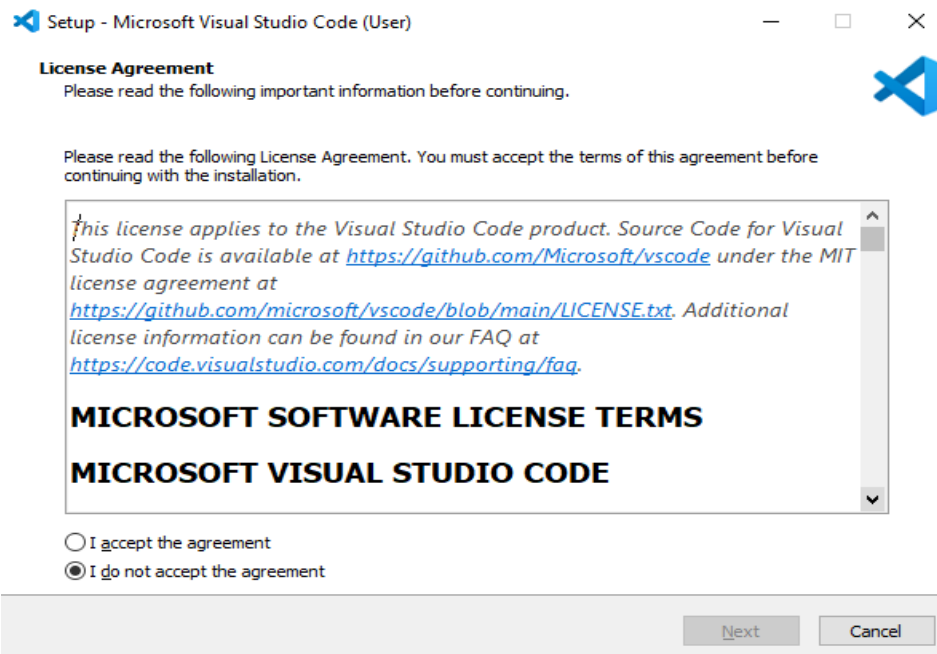
Download the python idle 3.10.4 and link is given below.

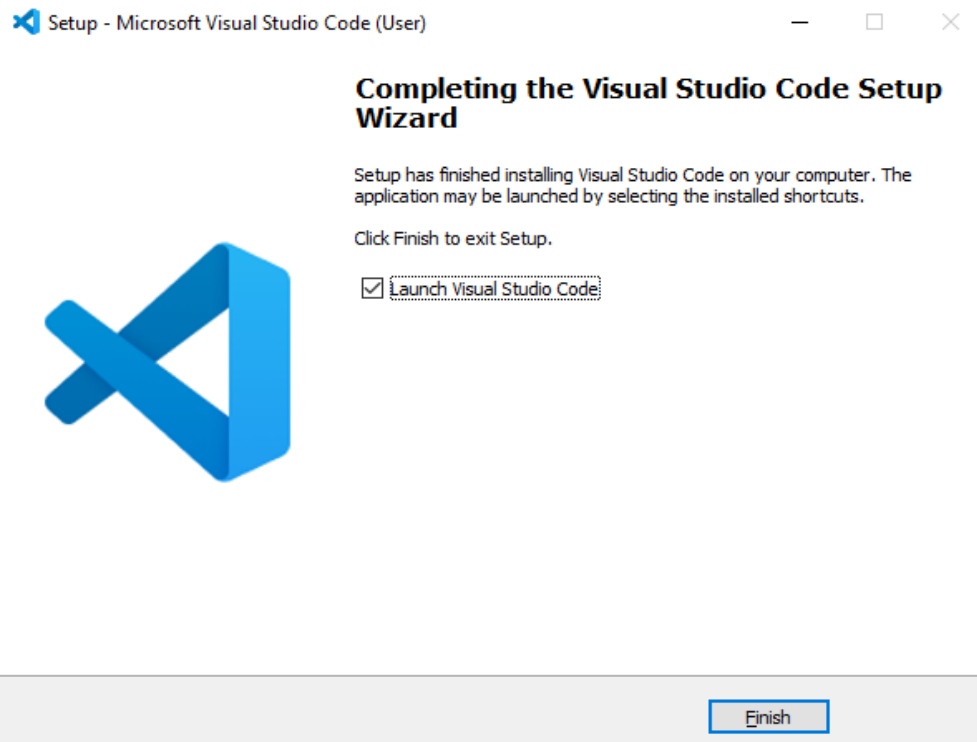
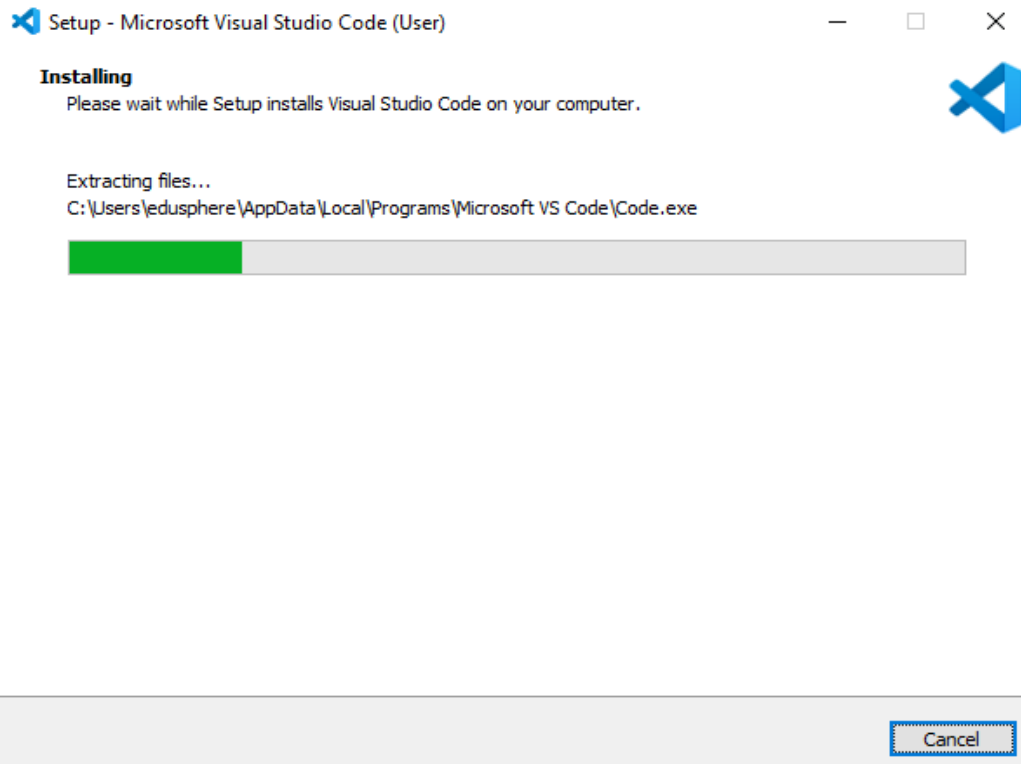
<https://www.python.org/downloads/release/python-3104/>



Click install and process the initialization.

4.3.2 VISUAL STUDIO CODE:





4.3.3 NUMPY:

```
C:\Users\acer>pip install numpy
Collecting numpy
  Downloading https://files.pythonhosted.org/packages/ce/ad/2e88f36b56f64f70c081b32fa5512dacedf12005ccb0c2d300d44dcc1215/numpy-1.17.4-cp37-cp37m-win32.whl (10.7MB)
    | 10.7MB 467kB/s
Installing collected packages: numpy
Successfully installed numpy-1.17.4
```

Pip Install Numpy.

4.3.4 TENSOR FLOW:

```
(tensor_env) kumar_satyam@satyam:~/gfg/tensor$ pip install --upgrade tensorflow
Collecting tensorflow
  Using cached tensorflow-2.5.0-cp36-cp36m-manylinux2010_x86_64.whl (454.3 MB)
Collecting numpy~=1.19.2
  Using cached numpy-1.19.5-cp36-cp36m-manylinux2010_x86_64.whl (14.8 MB)
Collecting wheel~=0.35
  Using cached wheel-0.36.2-py2.py3-none-any.whl (35 kB)
Collecting keras-nightly~=2.5.0.dev
  Using cached keras_nightly-2.5.0.dev2021032900-py2.py3-none-any.whl (1.2 MB)
Collecting h5py~=3.1.0
  Using cached h5py-3.1.0-cp36-cp36m-manylinux1_x86_64.whl (4.0 MB)
Collecting six~=1.15.0
  Using cached six-1.15.0-py2.py3-none-any.whl (10 kB)
Collecting grpcio~=1.34.0
  Using cached grpcio-1.34.1-cp36-cp36m-manylinux2014_x86_64.whl (4.0 MB)
```

Pip Install TensorFlow.

4.3.5 PANDAS:

```
Microsoft Windows [Version 10.0.19043.1083]
(c) Microsoft Corporation. All rights reserved.

C:\Users\>pip install pandas
Collecting pandas
  Downloading https://files.pythonhosted.org/packages/cb/e3/c0bcefb1b3835564f69094135de105a3def2eeb2689338a906bfc659c99d0/pandas-1.3.0-cp38-cp38-win_amd64.whl (10.2MB)
    | 10.2MB 3.3MB/s
Collecting pytz>=2017.3 (from pandas)
  Downloading https://files.pythonhosted.org/packages/70/94/784178ca5dd892a98f113cdd923372024dc04b8d40abe77ca76b5fb90ca6/pytz-2021.1-py2.py3-none-any.whl (510kB)
    | 512kB 6.4MB/s
Collecting python-dateutil>=2.7.3 (from pandas)
  Downloading https://files.pythonhosted.org/packages/36/7a/87837f39d0296e723bb9b62bbb257d035c7f6128853c78955f57342a56d/python_dateutil-2.8.2-py2.py3-none-any.whl (247kB)
    | 256kB ...
Collecting numpy>=1.17.3 (from pandas)
  Downloading https://files.pythonhosted.org/packages/df/22/b74e5cedeeef1e3f108c986bd0b75600997d8b25def334a68f08d372db523/numpy-1.21.0-cp38-cp38-win_amd64.whl (14.0MB)
    | 14.0MB 2.2MB/s
Collecting six>=1.5 (from python-dateutil>=2.7.3->pandas)
  Downloading https://files.pythonhosted.org/packages/d9/5a/e7c31adbe875f2abbb91bd84cf2dc52d792b5a01506781dbcf25c91daf11/six-1.16.0-py2.py3-none-any.whl
Installing collected packages: pytz, six, python-dateutil, numpy, pandas
Successfully installed numpy-1.21.0 pandas-1.3.0 python-dateutil-2.8.2 pytz-2021.1 six-1.16.0
WARNING: You are using pip version 19.2.3, however version 21.1.3 is available.
You should consider upgrading via the 'python -m pip install --upgrade pip' command.
```

Pip Install Pandas.

4.3.6 SCIKIT-LEARN:

```
C:\Users\Geeks>pip install scikit-learn
Collecting scikit-learn
  Downloading scikit_learn-0.24.2-cp38-cp38-win_amd64.whl (6.9 MB)
    | 6.9 MB 6.8 MB/s
Requirement already satisfied: joblib>=0.11 in c:\users\geeks\anaconda3\lib\site-packages (from scikit-learn) (1.0.1)
Requirement already satisfied: threadpoolctl>=2.0.0 in c:\users\geeks\anaconda3\lib\site-packages (from scikit-learn) (2.2.0)
Requirement already satisfied: scipy>=0.19.1 in c:\users\geeks\anaconda3\lib\site-packages (from scikit-learn) (1.7.1)
Requirement already satisfied: numpy>=1.13.3 in c:\users\geeks\anaconda3\lib\site-packages (from scikit-learn) (1.20.3)
Installing collected packages: scikit-learn
Successfully installed scikit-learn-0.24.2
```

Pip Install Scikit-Learn.

CHAPTER V

CONCLUSION

5.1 CONCLUSION:

Phase I of the project focuses on conducting an extensive literature survey and formulating the proposed work based on the identified research gaps. During this phase, existing studies related to helmet violation detection, traffic surveillance systems, and deep learning–based object detection techniques are thoroughly reviewed. The survey highlights that most traditional systems rely on manual monitoring or basic image processing methods, which are time-consuming, error-prone, and ineffective under real-world conditions such as poor lighting, complex backgrounds, and varying helmet styles. Recent research adopting deep learning models demonstrates improved accuracy, but several limitations remain, including limited real-time performance, lack of scalability, and insufficient integration with enforcement mechanisms. Based on these observations, the shortcomings of existing approaches are carefully analyzed to identify opportunities for improvement. As a result, the proposed work is designed to overcome these limitations by introducing a fully automated, real-time helmet violation detection framework. The proposed approach emphasizes high detection accuracy, robustness across diverse traffic scenarios, and seamless integration with alert and notification systems for effective enforcement. Phase I conclude with a clear problem definition, well-defined objectives, and a structured system design that forms the foundation for subsequent implementation and evaluation phases of the project.

5.2 PHASE II OF THE PROJECT:

Phase II of the project focuses on the practical implementation of the proposed helmet violation detection system and the validation of its real-time functionality. In this phase, the designed architecture is translated into a working system by integrating deep learning models with real-time video processing. Live or recorded CCTV footage is used as input, and each video frame is processed to detect motorcyclists and determine helmet usage. The trained detection model is deployed to accurately classify riders as helmeted or non-helmeted under various traffic and environmental conditions. When a helmet violation is identified, the system immediately captures the relevant frame as evidence and applies predefined violation logic. Based on this logic, notifications are automatically generated and sent to the concerned authorities, enabling prompt enforcement action. In addition, warning messages are issued to riders to increase awareness and discourage repeated violations. The notification mechanism ensures seamless communication without interrupting traffic flow. Phase II also involves testing the system for accuracy, response time, and reliability in real-world scenarios. Overall, this phase demonstrates the feasibility, efficiency, and effectiveness of the implemented system, confirming its capability to support automated enforcement and enhance road safety through timely helmet violation detection and notification.

REFERENCE:

- [1] İrfan Kılıç, Galip Aydin 2018, Turkish Vehicle License Plate Recognition Using Deep Learning, International Conference on Artificial Intelligence and Data Processing pp.1-5.
- [2] V Gnanaprakash 2021, Automatic number plate recognition using deep learning, to cite this articles 2021 IOP Conf. Ser.: Mater. Sci. Eng. 1084 012027.
- [3] R Shashidhar, AS Manjunath, R Santhosh Kumar, M Roopa, SB Puneeth 2021, Vehicle Number Plate Detection and Recognition using YOLO-V3 and OCR Method, IEEE International Conference on Mobile Networks and Wireless Communications (ICMNBC), 1-5.
- [4] DR Vedhaviyassh, R Sudhan, G Saranya, Mozhgan Safa, D Arun 2022, Comparative analysis of easy-ocr and tesseract-ocr for automatic license plate recognition using deep learning algorithm, 6th International Conference on Electronics, Communication and Aerospace Technology, 966-971.
- [5] Mohamed Al-Mheiri, Omar Kais, Talal Bonny 2022, Car Plate Recognition Using Machine Learning, Advances in Science and Engineering Technology International Conferences (ASET), 1-6.
- [6] N. Saleem, H. Muazzam, H.M. Tahir and U. Farooq, "Automatic License Plate Recognition using Extracted Features", Proceedings of 4th International Symposium on Computational and Business Intelligence, pp. 221-225, 2016.
- [7] R. Islam, K.F. Sharif and S. Biswas, "Automatic Vehicle Number Plate Recognition using Structured Elements", Proceedings of IEEE International Conference on Systems, Process and Control, pp. 44-48, 2015.
- [8] P. Prabhakar, P. Anupama and S.R. Resmi, "Automatic Vehicle Number Plate Detection and Recognition", Proceedings of IEEE International Conference on Control, Instrumentation, Communication and Computational Technologies, pp. 185-190, 2014.
- [9] J. Chong, C. Tianhua and J. Linhao, "License Plate Recognition based on Edge Detection Algorithm", Proceedings of 9th IEEE International Conference on Intelligent Information Hiding and Multimedia Signal Processing, pp. 395-398, 2013.
- [10] K.M. Hung and C.T. Hsieh, "A Real-Time Mobile Vehicle License Plate Detection and Recognition", Tamkang Journal of Science and Engineering, Vol. 13, No. 4, pp. 433-442, 2010.
- [11] A. Puranic, K.T. Deepak and V. Umadevi, "Vehicle Number Plate Recognition System: A Literature Review and Implementation using Template Matching", International Journal of Computer Applications, Vol. 134, No. 1, pp. 12-16, 2016.
- [12] P. Sai Krishna, "Automatic Number Plate Recognition by using MATLAB", International Journal of Innovative Research in Electronics and Communications, Vol. 2, No. 4, pp. 1-7, 2015.

[13] C. Arth, C. Leistner, and H. Bischof. Tricam: An Embedded Platform for Remote Traffic Surveillance. In Proceedings of the IEEE Conference on Computer Vision and Pattern.